HIP-0302: Hanzo Replicate

Encrypted SQLite Durability for Base Services

Zach Kelling

Hanzo AI

April 2026

HIP	0302
Title	Hanzo Replicate — Encrypted SQLite Durability for Base Services
Author	Zach Kelling (`zach@hanzo.ai`)
Type	Standards Track
Category	Infrastructure
Status	Draft
Created	2026-04-09
Requires	HIP-0027 (KMS), HIP-0032 (Object Storage), HIP-0065 (Backup & DR)

Abstract

This proposal specifies how all Hanzo Base-powered services achieve durable state persistence through continuous encrypted WAL replication to S3-compatible storage. The system comprises two packages: `hanzoai/replicate` (a rebranded Litestream with age encryption support) and `hanzoai/age` (X25519/Curve25519 file encryption). Every Base service—IAM, KMS, ATS, BD, TA, Tasks, and all future services—deploys `replicate` as a Kubernetes sidecar that continuously streams encrypted WAL frames to `hanzoai/s3`. The result is sub-second RPO, seconds-scale RTO, and zero dependence on centralized databases or persistent volume durability.

1 Motivation

Hanzo Base (HIP Base Runtime) packages an encrypted SQLite database per organization into a single Go binary. This architecture delivers sub-millisecond CRUD latency, per-tenant isolation by construction, and zero external database dependencies. It is the right architecture for sovereign applications.

It has one gap: durability on ephemeral infrastructure.

Kubernetes pods are ephemeral. A node drain, an OOM kill, a failed health check, or a routine deployment all destroy local state. Current Base deployments use either `emptyDir` volumes (state lost on every restart) or PersistentVolumeClaims (state survives restart but not node failure, and PVCs introduce scheduling constraints that conflict with horizontal scaling).

The consequences are concrete:

1. **Data loss on restart.** An `emptyDir` deployment of ATS loses all trade records, order history, and portfolio state when the pod restarts. This is unacceptable for a regulated securities exchange.
2. **Node affinity locks.** A PVC-backed deployment pins a pod to one node. If that node fails, the pod cannot reschedule until the volume is manually detached or the cloud provider's volume controller completes failover—typically 2–10 minutes on GKE.
3. **No disaster recovery.** Neither `emptyDir` nor PVC provides off-site backup. A storage controller bug, a zone outage, or accidental `kubectl delete pvc` destroys all state with no recovery path.
4. **No cross-region replication.** PVCs are zone-scoped. A zone failure takes down both the pod and its data. Regional redundancy requires an external replication mechanism.

This HIP solves all four problems with a single mechanism: continuous WAL replication to S3-compatible object storage, encrypted with age before leaving the pod.

2 Background

2.1 Hanzo Base Architecture

Hanzo Base runs one SQLite database per organization. Each database is encrypted at rest with sqlcipher (AES-256-CBC, PBKDF2-derived key). The encryption key is stored in Hanzo KMS (HIP-0027) and fetched at startup. Base exposes a REST API, a realtime SSE subscription layer, and a JavaScript plugin runtime (Goja VM). All data operations go through SQLite's WAL (Write-Ahead Log) mode.

WAL mode is critical to this proposal. In WAL mode, SQLite writes all changes to a separate WAL file before checkpointing them into the main database. This creates a sequential, append-only stream of changes—exactly the primitive that continuous replication requires.

2.2 Litestream

Litestream [1] is an open-source tool by Ben Johnson that continuously replicates SQLite databases to S3-compatible storage. It works by monitoring the SQLite WAL file and streaming new frames to a replica as they are written. On restore, it downloads the most recent snapshot and replays WAL frames to reconstruct the database to its last replicated state.

Litestream operates as a separate process alongside the application. It requires no application code changes, no special SQLite build, and no coordination protocol. It reads the WAL file directly from the filesystem.

2.3 Beeper's Encryption Fork

Beeper (acquired by Automattic, 2024) forked Litestream to add age encryption for their Matrix bridge infrastructure. Their fork encrypts each WAL segment and snapshot with an age recipient before uploading to S3. This ensures that the S3 operator (or any party with bucket access) cannot read the replicated data without the age identity (private key).

Beeper's fork demonstrated the viability of encrypted Litestream replication in production at scale (millions of Matrix bridge databases). However, it was never upstreamed and has diverged from mainline Litestream.

2.4 age Encryption

age [2] is a file encryption tool by Filippo Valsorda designed as a modern replacement for PGP. It uses X25519 for key agreement, ChaCha20-Poly1305 for authenticated encryption, and HKDF for key derivation. age has three properties that make it suitable for this use case:

1. **Small keys.** An age identity (private key) is a single 62-character string. A recipient (public key) is a single 59-character string. Both fit in a Kubernetes Secret without base64 bloat.
2. **Composable recipients.** A file can be encrypted to multiple recipients. This supports key rotation (encrypt to both old and new keys) and escrow (encrypt to both service key and recovery key).
3. **No configuration.** age has no cipher suites, no key formats, no modes. There is one algorithm. There is one way.

3 Specification

3.1 Components

The replication system consists of three components:

Component	Image	Purpose
`hanzoai/replicate`	`ghcr.io/hanzoai/replicate`	WAL replication + age encryption
`hanzoai/age`	`ghcr.io/hanzoai/age`	X25519 key generation
`hanzoai/s3`	`ghcr.io/hanzoai/s3`	S3-compatible object storage

Table 1: System components.

`hanzoai/replicate` is a fork of Litestream incorporating Beeper's age encryption patches, rebased onto mainline Litestream, and maintained as part of the Hanzo ecosystem. The binary is called `replicate`. It accepts the same configuration format as Litestream with one addition: an `age` block specifying identities and recipients.

`hanzoai/age` is a container image packaging the `age-keygen` binary for key generation in init containers and CI pipelines.

`hanzoai/s3` (HIP-0032) provides the S3-compatible storage backend.

3.2 Replication Protocol

The replication lifecycle for a single SQLite database:

1. **Startup.** `replicate` reads the configuration file, connects to the S3 endpoint, and checks for an existing replica.
2. **Restore (if needed).** If the local database does not exist and a replica exists in S3, `replicate` downloads the most recent snapshot, decrypts it with the age identity, replays all subsequent WAL segments (each decrypted individually), and writes the restored database to the configured path.
3. **Monitor.** `replicate` monitors the SQLite WAL file using `inotify` (Linux) or `kqueue` (macOS). When new frames are written, it reads them from the WAL.
4. **Encrypt.** Each WAL segment is encrypted with the configured age recipients before upload. Encryption happens in memory. Plaintext WAL data never touches S3.
5. **Upload.** The encrypted WAL segment is uploaded to S3 at the configured path. The upload is atomic (multipart with commit).
6. **Snapshot.** Periodically (default: every 24 hours), `replicate` takes a full snapshot of the database, encrypts it, and uploads it to S3. Old snapshots are retained according to the configured retention policy.
7. **Shutdown.** On SIGTERM, `replicate` flushes any pending WAL frames, uploads them, and exits cleanly. Kubernetes sends SIGTERM before SIGKILL, so graceful shutdown is the normal path.

3.3 Encryption Layers

Data is protected by three independent encryption layers:

Layer	Mechanism	Scope	Protects Against
1. Disk	sqlcipher (AES-256-CBC)	Database at rest	Local filesystem access
2. S3	age (X25519 + ChaCha20)	Replicas in storage	S3 operator, bucket leak
3. Wire	TLS 1.3	In transit	Network interception

Table 2: Encryption layers. Each layer is independent—compromise of one does not compromise the others.

Layer 1 (sqlcipher) is managed by Hanzo Base. Layer 2 (age) is managed by `replicate`. Layer 3 (TLS) is managed by the S3 endpoint. This HIP specifies Layer 2.

3.4 Recovery Objectives

Metric	Target	Mechanism
RPO (Recovery Point Objective)	$\leq$ 1 second	Continuous WAL streaming
RTO (Recovery Time Objective)	$\leq$ 30 seconds	Snapshot restore + WAL replay
Retention	72 hours WAL, 30 days snapshots	S3 lifecycle policy

Table 3: Recovery objectives.

The 1-second RPO is achieved by the default `sync-interval` of 1 second. WAL frames generated within the last sync interval may be lost on unclean shutdown (SIGKILL without preceding SIGTERM). In practice, Kubernetes provides a 30-second grace period between SIGTERM and SIGKILL, so data loss on clean shutdown is zero.

The 30-second RTO accounts for S3 download latency for the most recent snapshot (typically 1–5 seconds for databases under 1 GiB) plus WAL replay time (linear in the number of frames since the last snapshot).

4 Configuration

4.1 Replicate Configuration File

The configuration file follows the Litestream YAML format with the addition of age encryption fields.

```yaml
dbs:
  - path: /data/base/data.db
    replicas:
      - type: s3
        bucket: hanzo-replicate
        path: ${SERVICE}/${NAMESPACE}/${POD_NAME}
        endpoint: s3.hanzo.ai
        region: us-east-1
        sync-interval: 1s
        snapshot-interval: 24h
        retention: 72h
        retention-check-interval: 1h
        age:
          identities: /secrets/age/identity.txt
```

```
        recipients: /secrets/age/recipients.txt
```

Listing 1: replicate.yml — canonical configuration.

4.2 Environment Variables

Variable	Required	Purpose
LITESTREAM_ACCESS_KEY_ID	Yes	S3 access key
LITESTREAM_SECRET_ACCESS_KEY	Yes	S3 secret key
SERVICE	Yes	Service name (ats, iam, kms, ...)
NAMESPACE	Yes	K8s namespace
POD_NAME	Yes	K8s pod name (from fieldRef)

Table 4: Required environment variables.

S3 credentials are stored in Hanzo KMS (HIP-0027) and injected as Kubernetes Secrets. They are never committed to manifests.

4.3 S3 Path Structure

Replicas are organized by service, namespace, and pod:

```
s3://hanzo-replicate/
  ats/liquidity/ats-0/
    generations/
      00000001/          # snapshot + WAL segments
      00000002/
  iam/liquidity/iam-0/
    generations/
      00000001/
  kms/liquidity/kms-0/
    generations/
      00000001/
```

Listing 2: S3 path layout.

The three-level prefix (`service/namespace/pod`) ensures per-tenant isolation. No service can read another service's replicas without the corresponding age identity.

5 Key Management

5.1 Key Hierarchy

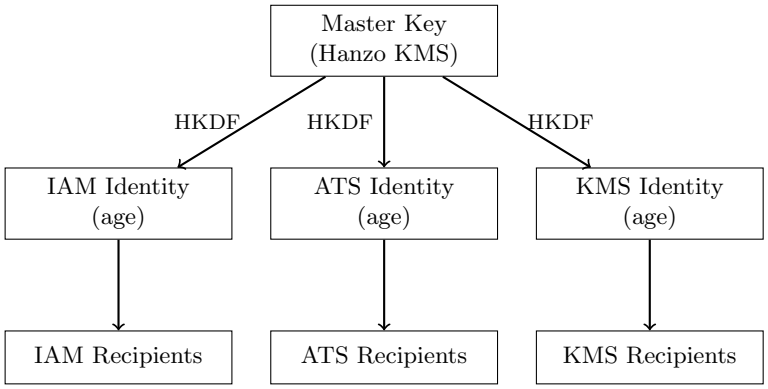

Figure 1: Key derivation hierarchy. Per-service age identities are derived from the master key via HKDF.

1. **Master encryption key.** Stored in Hanzo KMS (HIP-0027). This is the root of trust for all replicate encryption. It is a 256-bit symmetric key used as HKDF input keying material.
2. **Per-service age keypairs.** Derived from the master key via HKDF with the service name as the info parameter: `HKDF-SHA256(master, salt="hanzo-replicate", info=service_name)`. The derived 32 bytes are used as the X25519 private key (age identity). The corresponding public key is the age recipient.
3. **Recipients file.** Contains the public keys (recipients) of all parties authorized to decrypt the replica. For normal operation, this is the service's own recipient. For disaster recovery, this includes a recovery recipient held offline.

5.2 Key Rotation

Key rotation proceeds without downtime:

1. Generate a new age keypair (derived from a new HKDF info or a rotated master key).
2. Add the new recipient to the recipients file. The old recipient remains.
3. Restart the `replicate` sidecar. New WAL segments and snapshots are encrypted to both old and new recipients.
4. After the retention period expires (72 hours for WAL, 30 days for snapshots), all surviving replicas are encrypted to the new recipient.
5. Remove the old recipient from the recipients file.
6. The old identity is retained in KMS for historical reads. It is marked as `rotated` but never deleted.

5.3 Emergency Recovery

Full restore requires two keys: the master key from Hanzo KMS and the per-service age identity derived from it. The recovery procedure:

1. Retrieve the master key from KMS (requires KMS admin authentication).
2. Derive the service identity: `age-keygen -o identity.txt` from the HKDF output.
3. Run `replicate restore -config replicate.yml /data/base/data.db`.

4. The restored database is encrypted with sqlcipher. The sqlcipher key is also in KMS.

If KMS itself is unavailable (catastrophic failure), the KMS database is restored first using the offline recovery identity (stored in a physical safe or HSM).

6 Deployment

6.1 Kubernetes Sidecar Pattern

Every Base service deploys `replicate` as an init container (restore) and a sidecar container (continuous replication).

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: ats
spec:
  template:
    spec:
      initContainers:
        - name: replicate-restore
          image: ghcr.io/hanzoai/replicate:latest
          args:
            - restore
            - -if-db-not-exists
            - -if-replica-exists
            - -config
            - /etc/replicate/replicate.yml
            - /data/base/data.db
          volumeMounts:
            - name: data
              mountPath: /data/base
            - name: replicate-config
              mountPath: /etc/replicate
            - name: age-secrets
              mountPath: /secrets/age
              readOnly: true
          resources:
            requests: { memory: 16Mi, cpu: 10m }
            limits:   { memory: 64Mi, cpu: 100m }
      containers:
        - name: base
          image: ghcr.io/hanzoai/ats:latest
          # ... service container
          volumeMounts:
            - name: data
              mountPath: /data/base
        - name: replicate
          image: ghcr.io/hanzoai/replicate:latest
          args:
            - replicate
            - -config
            - /etc/replicate/replicate.yml
          volumeMounts:
            - name: data
              mountPath: /data/base
            - name: replicate-config
              mountPath: /etc/replicate
```

```
            - name: age-secrets
              mountPath: /secrets/age
              readOnly: true
          env:
            - name: LITESTREAM_ACCESS_KEY_ID
              valueFrom:
                secretKeyRef:
                  name: s3-credentials
                  key: access-key-id
            - name: LITESTREAM_SECRET_ACCESS_KEY
              valueFrom:
                secretKeyRef:
                  name: s3-credentials
                  key: secret-access-key
          resources:
            requests: { memory: 16Mi, cpu: 10m }
            limits:   { memory: 64Mi, cpu: 100m }
      volumes:
        - name: data
          emptyDir: {}
        - name: replicate-config
          configMap:
            name: replicate-config
        - name: age-secrets
          secret:
            secretName: replicate-age-keys
```

Listing 3: Kubernetes pod spec (abbreviated).

Note: the `data` volume is `emptyDir`. PVCs are no longer required. On pod startup, the init container restores the database from S3. During operation, the sidecar continuously replicates. On pod termination, the sidecar flushes pending WAL frames before exiting.

6.2 Resource Requirements

Container	CPU Req	CPU Lim	Mem Req	Mem Lim
replicate-restore (init)	10m	100m	16Mi	64Mi
replicate (sidecar)	10m	100m	16Mi	64Mi

Table 5: Resource requirements. The sidecar is lightweight—it reads the WAL file and uploads compressed, encrypted chunks.

6.3 Multi-Database Services

Services with per-org databases (ATS, BD, TA) replicate multiple SQLite files. The `replicate.yml` configuration supports multiple `dbs` entries. A filesystem watcher discovers new databases created at runtime and adds them to the replication set dynamically.

```
dbs:
  - path: /data/base/data.db
    replicas:
      - type: s3
        bucket: hanzo-replicate
        path: ats/liquidity/ats-0/system
        # ... same age config
```

```
  - path: /data/base/orgs/*.db
    replicas:
      - type: s3
        bucket: hanzo-replicate
        path: ats/liquidity/ats-0/orgs
        # ... same age config
```

Listing 4: Multi-database configuration.

7 Affected Services

Service	Database(s)	Typical Size	PVC	Replicate
IAM	iam.db	50–200 MiB	1 GiB	Yes
KMS	kms.db (sqlcipher)	10–50 MiB	1 GiB	Yes
ATS	data.db + per-org .db	1–20 GiB	20 GiB	Yes
BD	data.db + per-org .db	1–10 GiB	20 GiB	Yes
TA	data.db + per-org .db	500 MiB–5 GiB	20 GiB	Yes
Tasks	tasks.db	100–500 MiB	5 GiB	Yes

Table 6: Affected services. The PVC column shows the current allocation; with replicate, PVCs are replaced by `emptyDir` and the listed sizes become the S3 storage consumption.

All services listed above are **required** to deploy the replicate sidecar. New Base services created after this HIP is finalized inherit the requirement automatically.

8 Security Analysis

8.1 Threat Model

Threat	Mitigation	Layer
S3 bucket compromise	age encryption	Layer 2
S3 operator reads replicas	age encryption	Layer 2
Network interception	TLS 1.3 to S3 endpoint	Layer 3
Pod filesystem access	sqlcipher encryption	Layer 1
Age identity theft	KMS access control, HKDF	Key mgmt
Key loss	Offline recovery identity	Key mgmt
Replay attack on WAL	Generation counters, HMAC	Protocol

Table 7: Threat model and mitigations.

8.2 Isolation Guarantees

Per-org isolation is maintained at three levels:

1. **Database level.** Each org has a separate SQLite file with a separate sqlcipher key. Compromise of one org's sqlcipher key does not expose another org's data.
2. **S3 path level.** Each org's replica is stored at a distinct S3 prefix. S3 bucket policies restrict access by prefix.

3. **Encryption level.** Each service has its own age identity. The ATS age identity cannot decrypt IAM replicas. Cross-service access requires the master key.

8.3 What This HIP Does Not Cover

- **Application-level backup testing.** HIP-0065 specifies automated restore verification. This HIP defers to HIP-0065 for backup testing schedules and alerting.
- **Cross-region replication.** S3 bucket replication to a second region is an operational concern configured at the `hanzoai/s3` level (HIP-0032), not at the replicate level.
- **PostgreSQL replication.** Services using PostgreSQL (HIP-0029) use WAL-G or pgBackRest. This HIP applies only to SQLite databases managed by Hanzo Base.

9 Migration

9.1 Existing PVC-Backed Services

Services currently using PVCs migrate to replicate in three steps:

1. **Bootstrap.** Deploy the replicate sidecar alongside the existing PVC-backed pod. The sidecar begins replicating the PVC-resident database to S3. Wait for the initial snapshot to complete.
2. **Cutover.** Redeploy the StatefulSet with `emptyDir` instead of PVC. The init container restores from S3. Verify the restored database matches the PVC-resident database (row count, checksum).
3. **Cleanup.** Delete the PVCs. Update the service's Kustomize overlay to remove PVC-related resources.

9.2 New Services

New Base services created after this HIP include the replicate sidecar in their initial Kustomize base. No migration required.

10 Operational Concerns

10.1 Monitoring

`replicate` exposes Prometheus metrics on port 9090:

Metric	Description
`replicate_wal_bytes_total`	Total WAL bytes replicated
`replicate_wal_segments_total`	Total WAL segments uploaded
`replicate_snapshot_bytes_total`	Total snapshot bytes uploaded
`replicate_restore_duration_seconds`	Last restore duration
`replicate_replication_lag_seconds`	Time since last successful upload

Table 8: Prometheus metrics.

An alert fires if `replicate_replication_lag_seconds` exceeds 10 seconds for any service.

10.2 Failure Modes

- **S3 unavailable.** `replicate` buffers WAL frames locally and retries with exponential backoff. The application continues operating. RPO degrades to the duration of the S3 outage.
- **Age identity missing.** The init container fails to restore. The pod enters CrashLoopBackOff. Alert fires. Operator provisions the identity from KMS.
- **Corrupt replica.** `replicate` validates snapshots via checksum on restore. A corrupt snapshot triggers fallback to the previous generation.

11 Post-Quantum Security

The Hanzo ecosystem has achieved comprehensive post-quantum (PQ) coverage across all cryptographic layers. All previously planned mitigations are now deployed.

11.1 PQ Scorecard

Layer	Algorithm	PQ Status	Status
Disk encryption	AES-256 (sqlcipher)	Safe (128-bit PQ)	Deployed
HKDF derivation	HKDF-SHA-256	Safe (128-bit PQ)	Deployed
Field encryption	AES-256-GCM	Safe (128-bit PQ)	Deployed
S3 backup	age ML-KEM-768+X25519 (`age1pq`)	Safe (FIPS 203)	Deployed
TLS	X25519MLKEM768 (first curve)	Safe (hybrid PQ)	Deployed
JWT signing	ML-DSA-65 (FIPS 204)	Safe (Module-LWE+SIS)	Deployed
JWKS validation	ML-DSA-65	Safe	Deployed
MPC transport	Hybrid KEM (X25519+ML-KEM-768)	Safe	Deployed
MPC key shares	PQ KEM encryption	Safe	Deployed
Master keys	Cloud HSM (FIPS 140-2 L3)	Hardware isolation	Deployed
Chain signing	secp256k1 ECDSA	**Not PQ-safe**	EVM constraint

Table 9: Complete post-quantum scorecard. All layers PQ-safe except EVM chain signing.

11.2 S3 Backup Encryption (Deployed)

All age encryption uses ML-KEM-768 + X25519 hybrid recipients (`age1pq` prefix). The file key is derived from both shared secrets: file_key = HKDF-SHA256($ss_1 \| ss_2$, `"age-hybrid-v1"`). Label enforcement prevents downgrade to classical-only recipients.

11.3 TLS Post-Quantum (Deployed)

All TLS 1.3 connections negotiate X25519MLKEM768 as the first curve. This provides PQ protection for all data in transit, including S3 uploads and inter-service communication.

11.4 JWT Signing with ML-DSA-65 (Deployed)

Hanzo IAM issues JWT tokens signed with ML-DSA-65 (FIPS 204). The JWKS endpoint exposes the ML-DSA-65 public key. All services validate JWTs using the PQ-safe signature. EUF-CMA security under Module-LWE and Module-SIS hardness assumptions.

11.5 MPC Transport and Key Shares (Deployed)

MPC node-to-node communication uses hybrid KEM (X25519 + ML-KEM-768). MPC key shares at rest are encrypted with PQ KEM encryption.

11.6 Cloud HSM for Master Keys (Deployed)

Master encryption keys are stored in Cloud HSM (GCP Cloud KMS, FIPS 140-2 Level 3 Cavium HSMs). The master key never leaves the HSM: wrap, unwrap, and sign operations execute inside the HSM. Three keyrings per ecosystem (devnet, testnet, mainnet), 12 keys total. Mainnet uses HSM protection level.

11.7 Harvest-Now-Decrypt-Later: Closed

The combination of ML-KEM-768 hybrid encryption (S3), X25519MLKEM768 TLS (transit), ML-DSA-65 JWT signing (auth), PQ KEM for MPC, and Cloud HSM for master keys closes the HNDL attack vector completely.

12 References

References

[1] B. Johnson, "Litestream — Streaming replication for SQLite," https://litestream.io, 2021.

[2] F. Valsorda, "age — A simple, modern and secure encryption tool," https://age-encryption.org, 2019.

[3] Zetetic LLC, "SQLCipher — Full Database Encryption for SQLite," https://www.zetetic.net/sqlcipher/, 2008.

[4] Beeper, "Litestream fork with age encryption," https://github.com/nicolo-ribaudo/litestream-age, 2023.

[5] Z. Kelling, "Hanzo Base: The Single-Binary Runtime for Sovereign Applications," Hanzo Industries, March 2026.

[6] Hanzo AI Team, "HIP-0027: Secrets Management Standard," 2025.

[7] Hanzo AI Team, "HIP-0032: Object Storage Standard," 2025.

[8] Hanzo AI Team, "HIP-0065: Backup & Disaster Recovery Standard," 2026.

[9] M. Connolly, "X-Wing: The Hybrid KEM You've Been Looking For," Internet-Draft, IETF CFRG, https://www.ietf.org/archive/id/draft-connolly-cfrg-xwing-kem-05.html, 2024.

[10] NIST, "Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)," FIPS 203, August 2024.